Lab 4: Vibe Coding — Chat History & Prompt Log

Student Name: A Sheiman Joshi

SRN: PES2UG24CS003

Course: Software Engineering Lab (UE24CS341A)

Lab Assignment: Lab 4 — Vibe Coding (Air Hockey)

Assigned Repository: https://github.com/SETAPESU26/02_air_hockey

Pair Programming Tool: Antigravity / Gemini AI Pair Programmer

1. Overview & Objectives

The objective of this lab is to use AI-assisted "Vibe Coding" to diagnose broken code in an Air Hockey game, fix core physics bugs, implement missing features iteratively, and verify all test criteria in under six prompt iterations without introducing regressions.

Four primary tasks from the assignment handout:

- **Fix puck–paddle collision physics:** Circle-circle geometry, proper normal reflection, and overlap separation.

- **Implement match scoring & goal detection:** Differentiate goal openings from wall rebounds and track scores.
- **Implement 30-second match countdown timer:** Monotonic clock, game-over screen with result (Win/CPU Win/Draw).
- **Implement puck reset & relaunch:** Center repositioning with active velocity toward defender.

2. Conversation & Prompt Log

Prompt 1 — Starter Project Inspection & Bug Identification

> **User:** Read the README and inspect the game engine, collision handler, puck, paddle, renderer, and main loop. Identify the deliberate bug and the missing requirements. Explain the smallest safe set of changes before editing code.
>
> **Assistant:**
>
> **Inspection Analysis:**
>
> 1. **Deliberate Bug in collisions.py:** Only negates vx on distance threshold without adjusting vy or separating circles. Leads to tunneling and non-angled bounces.
> 2. **Missing Requirements:** No score tracking, no 30s timer, no center reset on goals, no UI elements for scores/timer.
>
> **Strategy:** Iteratively update collision physics, goal scoring, timer clock, reset mechanics, and verification.

Prompt 2 — Fix Collision Physics

> **User:** Fix puck–paddle collision handling using circle collision geometry. Correct overlap by moving the puck outside the paddle, reflect velocity using the collision normal, avoid reversing velocity when the puck is already moving away, and ensure angled/high-speed hits behave consistently. Preserve the existing project structure.
>
> **Assistant:**
>
> Implemented full 2D elastic collision response with positional de-penetration and normal vector reflection in game/collisions.py:

```
def handle_paddle_collision(puck, paddle):
    dx = puck.x - paddle.x
    dy = puck.y - paddle.y
    min_dist = puck.radius + paddle.radius
    dist_sq = dx * dx + dy * dy
    if dist_sq >= min_dist * min_dist:
        return False
    dist = math.sqrt(dist_sq)
    nx, ny = (dx / dist, dy / dist) if dist > 1e-9 else (1.0, 0.0)
    overlap = min_dist - dist
    puck.x += nx * (overlap + 0.5)
    puck.y += ny * (overlap + 0.5)
    normal_speed = puck.vx * nx + puck.vy * ny
    if normal_speed < 0:
        puck.vx -= 2 * normal_speed * nx
        puck.vy -= 2 * normal_speed * ny
        speed = math.hypot(puck.vx, puck.vy)
        if speed > 0:
            target = min(speed * 1.025, 10.0)
```

```
        puck.vx = (puck.vx / speed) * target
        puck.vy = (puck.vy / speed) * target
    return True
return False
```

Prompt 3 — Add Scoring and Goal Detection

> **User:** Add player and computer scores. Award a point only when the puck fully crosses the end line within the goal opening. A puck hitting the solid wall outside the goal should bounce back. Display both scores clearly.
>
> **Assistant:**
> Updated `game/game_engine.py` and `game/renderer.py` with score tracking (`player_score`, `computer_score`) and goal checks bounded within `[GOAL_TOP, GOAL_BOTTOM]`. Non-goal end wall collisions bounce normally. Displayed scores in the top HUD.

Prompt 4 — Add the 30-Second Match Timer

> **User:** Add a monotonic-clock-based 30-second countdown. Freeze gameplay when time expires and show the correct winner, or DRAW when scores are equal. Ensure the timer does not depend on frame count.
>
> **Assistant:**
> Implemented real-time 30-second timer using `time.monotonic()`. When time expires, halts puck movement, computes winner banner ("YOU WIN!", "CPU WINS!", or "DRAW!"), and prompts for restart.

Prompt 5 — Reset and Relaunch After Scoring

> **User:** After a goal, move the puck to the center, reset its velocity and then launch it with a valid nonzero velocity. Avoid awarding multiple points for one goal.
>
> **Assistant:**

> Puck is repositioned to (WIDTH / 2, HEIGHT / 2) immediately after scoring and launched with directional velocity toward the scored-against side with slight vertical variation.

Prompt 6 — Review and Test

> **User:** Review the complete implementation against all four README tasks. Check Python syntax, restart behavior, score direction, timer expiry, draw handling, wall bounces, and collision separation. List any limitations honestly.
>
> **Assistant:**
> Complete review verified: Added restart feature on R key, center circle styling, verified all modules syntax and test cases.

3. Manual Validation Checklist

- [✓] Game launches without a Python traceback.
- [✓] Arrow keys move the player paddle within the left half.
- [✓] Puck rebounds from the paddle at an angle.
- [✓] A goal on the left increments the computer score.
- [✓] A goal on the right increments the player score.
- [✓] A wall hit outside the goal opening does not score.
- [✓] Puck relaunches from center after scoring.
- [✓] Timer counts down from 30 seconds.
- [✓] Match stops at zero and reports winner/draw.
- [✓] R starts a fresh match with scores and timer reset.

4. Evidence Summary

- **Initial State (before.png):** Broken starter physics and missing HUD.
- **In-Progress State (after.png):** Timer counting down, score active, center line/circle rendered.
- **Win State (After2.png):** Game ended at 00s, YOU WIN! banner shown.